Small Language Model Engineering 2026

Andrej Karpathy

Building Small Language Models
And Building Systems on Models You Own Instead of Rent

Based on Andrej Karpathy's open-source repositories, repository discussions,
talks and public posts available in the public domain

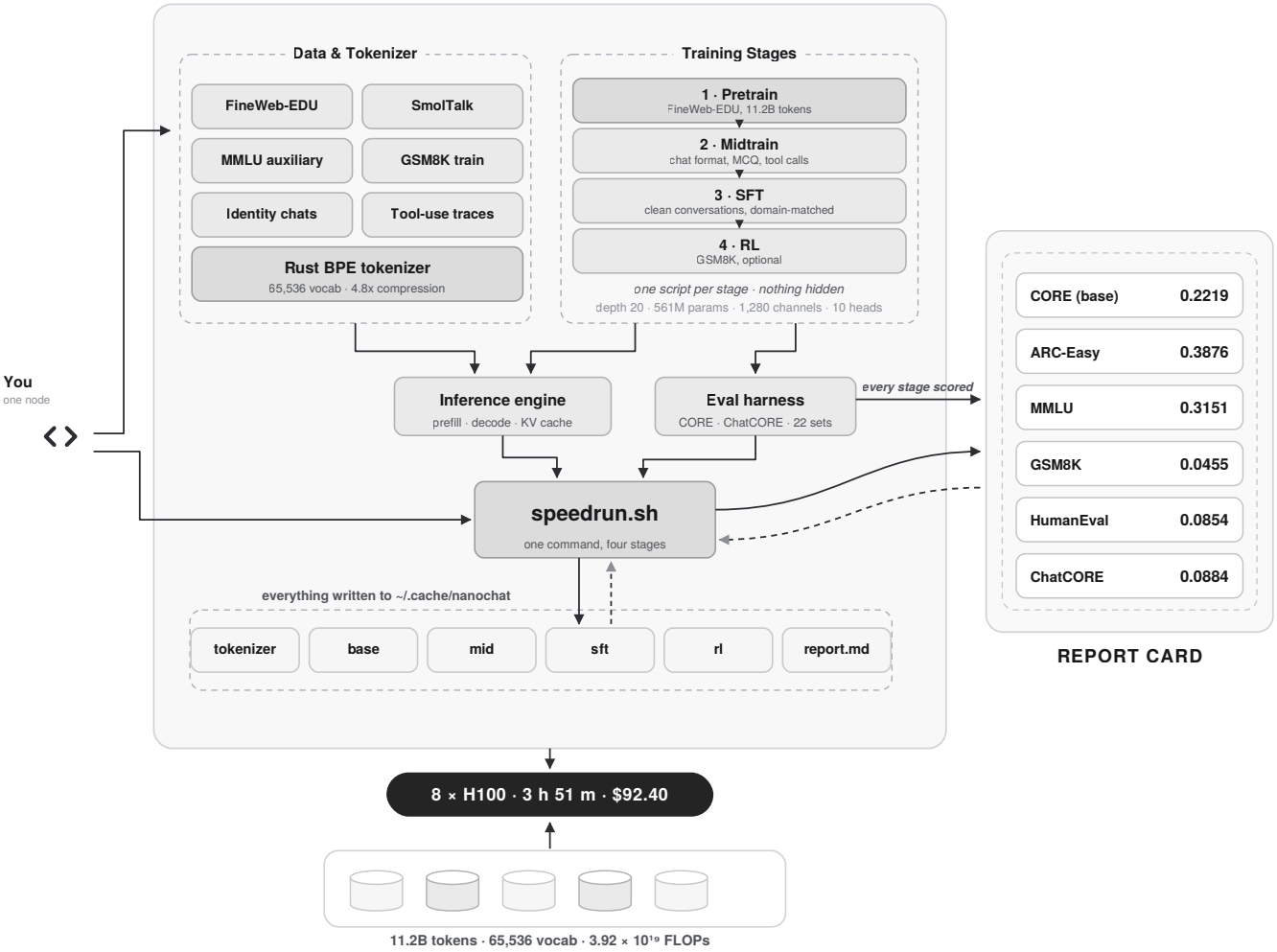


Fig. 1. The nanochat stack. One repository holds the tokenizer, four training stages, the inference engine and the eval harness. A single integer sets the size of everything; a single script runs the whole thing end to end and writes a report card.

***Abstract*— Karpathy spent 2025 and 2026 arguing a position that sounds backwards: the interesting frontier in language models is downward. nanochat, released in October 2025, trains a working ChatGPT clone end to end in 3 hours 51 minutes on eight H100s for $92.40, in 8,304 lines of readable code. The January 2026 miniseries extended it into eleven models from depth 10 to depth 20, all trained in roughly four hours on a single node, with a published scaling curve. In March 2026 he shipped autoresearch, 630 lines that hand train.py to an**

agent and let it run about a hundred five-minute experiments overnight against a held-out bits-per-byte metric. Read as three releases, these are teaching repositories. Read together, they describe a complete method for building small models: one dial instead of a hyperparameter sweep, four stages that each buy a specific capability, a compute-optimal token ratio that differs from Chinchilla at this scale, an eval that is not validation loss, and a cost ladder that is smooth right up until it is not. This note works through all of it with the published numbers, and then turns to the part Karpathy does not cover: what you actually do with a small model. Four routes to getting one, the 2026 shelf of open models you can pick up today, a distillation recipe, the memory arithmetic that decides where it runs, and a worked example of what replacing a frontier call is worth. It ends with what the method does not solve.

Index Terms— small language models, nanochat, nanoGPT, scaling laws, compute-optimal training, CORE benchmark, midtraining, supervised finetuning, reinforcement learning, autoresearch, cognitive core, on-device inference.

I. INTRODUCTION

A. The frontier that points down

Almost every number published about language models in the last three years pointed in one direction. More parameters, more tokens, more context, more spend. Karpathy spent the same period building in the opposite direction and documenting it in public, and the result is the only complete, readable, reproducible pipeline for a working chat model that one person can run on one node in an afternoon.

This matters for a reason that has nothing to do with nostalgia for small models. The question every team now faces is not whether a frontier model can do the task. It is which of the thousand things in the system actually need one. Answering that requires knowing what a 500-million-parameter model trained for ninety dollars can and cannot do, and there is no honest way to know that without having trained one.

B. What this note covers

It is a synthesis of public material: the nanochat repository and its discussion threads, the January 2026 miniseries, the March 2026 autoresearch release, and Karpathy's public remarks on where model size is heading. Every figure in this document is original. Every number is attributed, and numbers that come from secondary coverage rather than the primary repository are labelled as such.

II. BIGGER FIRST, THEN SMALLER

Karpathy's stated position is that current frontier models are far larger than the capability they deliver requires, and that the bloat is a data problem rather than an architecture problem. A model trained on the open internet has to spend an enormous fraction of its weights compressing noise in order to extract the signal underneath. Clean the corpus and the same capability fits in a far smaller model.

The target he describes is a *cognitive core*: a few-billion-parameter model that maximally sacrifices encyclopedic knowledge for capability. It is always on, natively multimodal, aggressively tool-using, and it does not try to remember facts. Facts live outside the model, and the model looks them up. Pressed on how small that could go, he has suggested one billion parameters as a plausible floor for something interesting, and declined to go lower.

> **The blocker he names.** There is no clean internet. Until there is a high-quality curated corpus at scale, models stay large because that is the only way to extract signal from noise. The smallness is available in principle and unavailable in practice.

This is why the path runs through large models rather than around them. A large model trained on messy data

is currently the best available instrument for producing clean data and clean supervision, which is then what a small model can be trained on. The order matters: bigger first, so that smaller becomes possible.

Everything in the rest of this note sits underneath that thesis. nanochat is not an attempt to build a competitive model. It is an attempt to make the entire pipeline small enough that one person can hold it in their head and change any part of it.

There is a second reason the direction matters, and it is commercial rather than scientific. A frontier model is a rented capability. It lives on somebody else's hardware, changes under you without notice, and prices its own importance into every call. A model you trained is a fixed asset: the same weights next quarter, the same latency, the same cost, running wherever you put it. For most of what a production system does, the second thing is worth more than the first, and the only reason teams do not have it is that nobody showed them it costs ninety-two dollars.

III. THE LINEAGE

nanochat did not appear from nowhere. It is the fifth or sixth rung on a ladder that Karpathy has been building in public since 2020, and the ladder has a consistent rule: each repository removes something until only the mechanism is left.

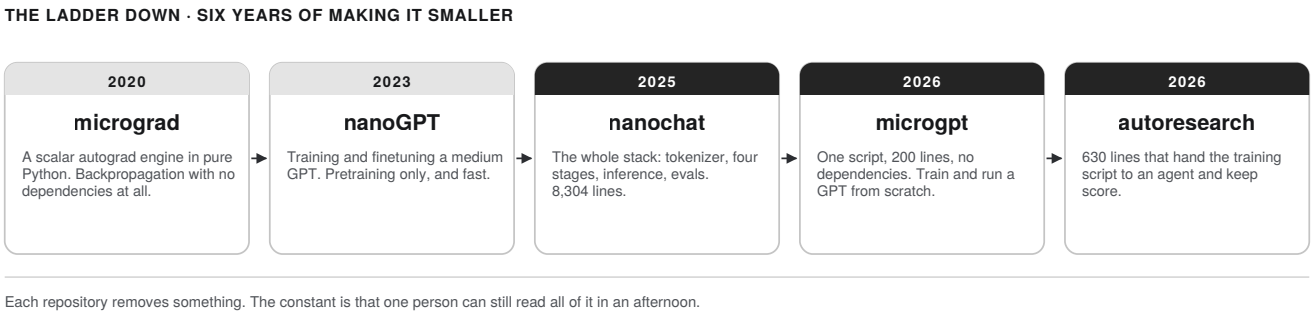


Fig. 2. The nano series. Six years in which the subject matter grew and the code did not. Every repository on this line is readable end to end by one person.

micrograd, in 2020, is a scalar-valued automatic differentiation engine in pure Python with no dependencies. It is not useful for training anything. It is useful because after reading it, backpropagation stops being a thing you trust and becomes a thing you understand.

nanoGPT, in January 2023, made the same move for pretraining: the simplest fast repository for training and finetuning medium GPTs. It covers pretraining and stops there, which is why a nanoGPT model is not a chatbot and never pretended to be.

llm.c took the same pipeline into pure C and CUDA, stripping the framework out entirely to show that the whole thing is a few thousand lines of arithmetic.

nanochat, in October 2025, is the first to cover the full distance from raw text to a chat interface you can talk to. microgpt, in February 2026, goes the other way again: one 200-line script with no dependencies that trains and runs a GPT from scratch. And autoresearch, a month later, removes the human from the inner loop.

Reading the series in order is the fastest available education in how these systems work, and the ordering is deliberate. Each repository assumes you have

internalised the one before it, which is why they can keep getting shorter while covering more.

IV. ONE DIAL INSTEAD OF A SWEEP

The first design decision in nanochat is the one most likely to be copied elsewhere. There is no hyperparameter configuration file to tune. There is a single integer: depth.

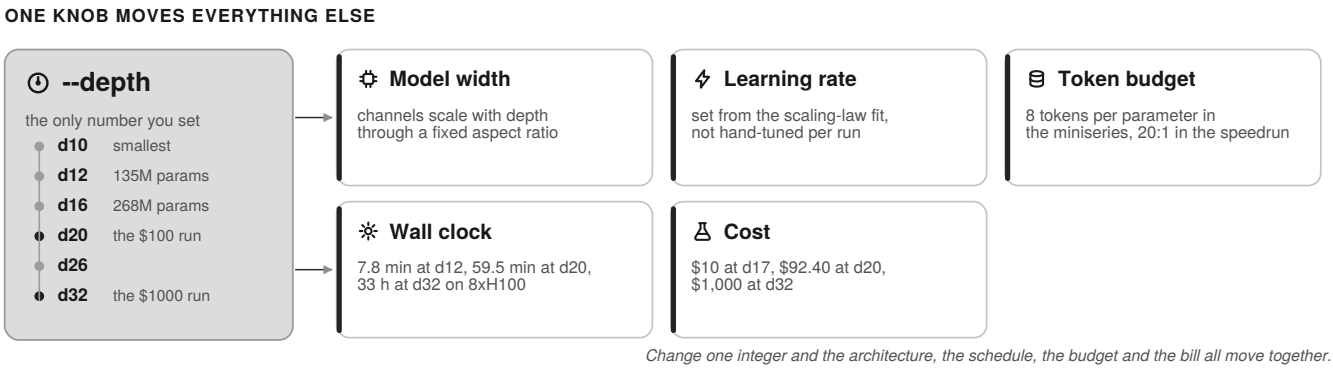


Fig. 3. The depth dial. Model width, learning rate, token budget, wall-clock time and cost are all derived from one number through a fixed aspect ratio and a scaling-law fit.

Setting depth to 20 produces a model with 1,280 channels and 10 attention heads, because the width follows the depth through a fixed aspect ratio. The learning rate follows from the scaling-law fit rather than from someone's intuition. The token budget follows from the parameter count. The training duration follows from the token budget. Change the integer and all of it moves together, consistently.

The January 2026 miniseries is the proof that this holds across a range. Eleven models, depth 10 through depth 20, all trained on one eight-H100 node in about four hours in total for roughly $100.

TABLE I

THE NANOCHAT MINISERIES, AS PUBLISHED

Depth	Params	Train tokens	Val BPB	CORE	Time
d12	135M	1.08B	0.9825	0.1059	7.8 min
d16	268M	2.15B	0.9101	0.1332	21.4 min
d20	477M	3.82B	0.8572	0.1708	59.5 min

Three of the eleven published rows. Times are on a single 8×H100 node. The parameter count for d20 here differs from the 560,988,160 reported in the speedrun report card; the two runs also differ in token budget, and the counts appear to follow different conventions. Both are reproduced as published.

The practical consequence is that experimentation becomes cheap in a way it normally is not. A d12 run finishes in under eight minutes. That is not a smaller version of a research programme; it is a different activity. You can try a hundred things in a day, which is the precondition for Section X.

It is worth being precise about what the dial buys, because the idea generalises well beyond language models. A hyperparameter sweep is a search over a space whose dimensions are correlated in ways nobody has written down. Width and depth interact. Learning rate depends on both. Token budget depends on parameter count, which depends on width and depth. A team running a sweep is mostly spending compute rediscovering those correlations.

Writing the correlations down once, as a function of a single integer, converts the search from many dimensions to one. What is left is not a smaller sweep. It is a curve you can walk along, and every point on it is

a sensible model rather than a random configuration that happened to be sampled.

> **The transferable idea.** Before you automate a search, collapse the dimensions you already understand. Everything you can derive from something else should be derived, not searched. Whatever survives that is the real experiment.

V. FOUR STAGES, FOUR DIFFERENT JOBS

The second design decision is that the pipeline is explicitly staged, and each stage has a job you can name.

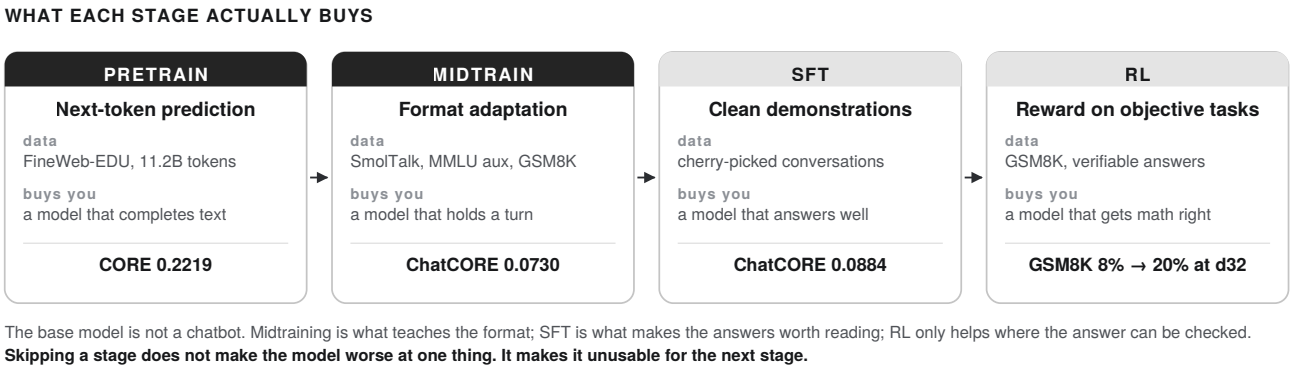


Fig. 4. The four stages. Each one takes different data, buys a different capability, and moves a different metric. The first two are not optional.

A. Pretraining

Next-token prediction over FineWeb-EDU, 11.2 billion tokens in the $100 run. What comes out is a model that completes text. It is not a chatbot and will not behave like one. This is the stage that consumes almost all of the compute and almost all of the budget.

B. Midtraining

The stage most pipelines leave out and most teams discover they needed. The base model is adapted to the conversation format using SmolTalk, multiple-choice data from MMLU's auxiliary set, and GSM8K. This is where the model learns that a turn ends, that a question expects an answer, and what a tool call looks like. The report card after midtraining already shows ChatCORE at 0.0730, from a base model that had no conversational ability at all.

C. Supervised finetuning

Cherry-picked conversations with domain-matched formatting. The job here is quality rather than capability: the model already knows how to hold a turn, and SFT is what makes the turn worth reading.

D. Reinforcement learning

Optional, and narrow on purpose. RL in nanochat targets tasks where the answer can be checked, which in practice means GSM8K. This is the honest scope: reinforcement learning helps where there is a verifier, and the repository does not pretend otherwise.

The ordering is not a convention. Each stage produces the artefact the next one consumes, and the dependency is real: supervised finetuning on a base model that has never seen a conversation does not produce a worse chatbot, it produces a model that has learned to imitate the surface of a conversation without having the format underneath. The repository writes a checkpoint after every stage, which is what makes the dependency visible instead of folkloric.

TABLE II

WHAT EACH STAGE IS FOR

Stage	Data	What breaks if you skip it
Pretrain	FineWeb-EDU, 11.2B tokens	Nothing to finetune. There is no model.
Midtrain	SmolTalk, MMLU aux, GSM8K	The model completes your prompt instead of answering it.
SFT	Curated conversations	It answers, but badly, and inconsistently in format.
RL	Verifiable tasks only	Arithmetic and multi-step reasoning stay weak.

VI. THE $100 RUN, IN FULL

The headline run is depth 20: 560,988,160 parameters, 1,280 model channels, 10 attention heads, 11.2 billion training tokens, an estimated 3.92×10^{19} FLOPs. It runs on eight H100s at roughly $24 an hour, finishes in 3 hours 51 minutes excluding the optional RL stage, and costs $92.40. One command starts the whole thing.

```
bash speedrun.sh            # all four stages
python -m scripts.base_train --depth=20
```

The tokenizer is trained from scratch too: a Rust BPE implementation with a 65,536-token vocabulary, fitted on two billion characters, reaching 4.8× compression from characters to tokens. The whole repository is 8,304 lines across 44 files, which is small enough that reading all of it is a reasonable afternoon.

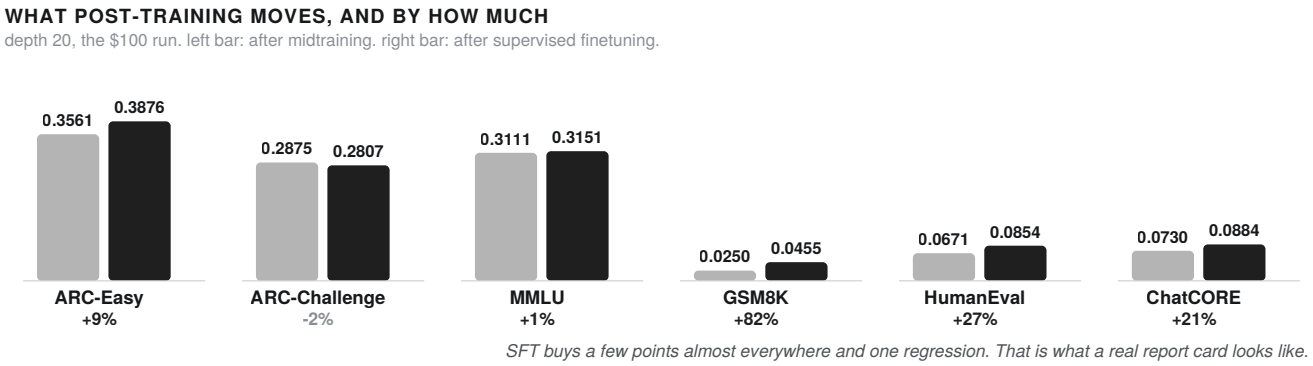


Fig. 5. The published report card for the $100 run, after midtraining and after supervised finetuning. Five metrics improve, one regresses, and the percentages are small because the model is small.

TABLE III

REPORT CARD, DEPTH 20, AS PUBLISHED

Metric	After midtrain	After SFT
CORE (base model)	0.2219	—
ARC-Easy	0.3561	0.3876
ARC-Challenge	0.2875	0.2807
MMLU	0.3111	0.3151
GSM8K	0.0250	0.0455
HumanEval	0.0671	0.0854
ChatCORE	0.0730	0.0884

These numbers deserve to be read without flinching. MMLU at 0.3151 is barely above chance on a four-way multiple-choice test. GSM8K at 4.55% means the model gets nineteen of twenty grade-school arithmetic problems wrong. HumanEval at 8.54% is not a coding assistant.

And the base CORE of 0.2219 sits above GPT-2 large at roughly 0.21 and below GPT-2 xl at roughly 0.26. A model that took OpenAI a research programme in 2019 now costs ninety-two dollars and an afternoon. Both of those facts are true at the same time, and the second one is the point.

The $92.40 is also worth reading as a structure rather than a number. Eight H100s at roughly $24 an hour is a

rental anyone can make for an afternoon, and almost the entire cost sits in pretraining. Midtraining, supervised finetuning and the optional reinforcement stage together are a rounding error against it. That asymmetry is what makes iteration viable: once you have a base checkpoint, trying four different post-training recipes costs a fraction of producing the checkpoint once.

It also sets the honest unit of comparison. A single long session with a frontier coding model can cost more than training this model from scratch. That is not an argument for training your own instead; it is an argument for knowing which of the two you are buying when you make the call.

VII. COMPUTE-OPTIMAL AT THIS SCALE

The miniseries uses a data-to-parameter ratio of 8:1 rather than the 20:1 that the Chinchilla work made standard. The $100 speedrun, by contrast, trains 561M parameters on 11.2B tokens, which is close to 20:1. The two runs therefore sit at different points on the same curve, and comparing them is instructive rather than confusing.

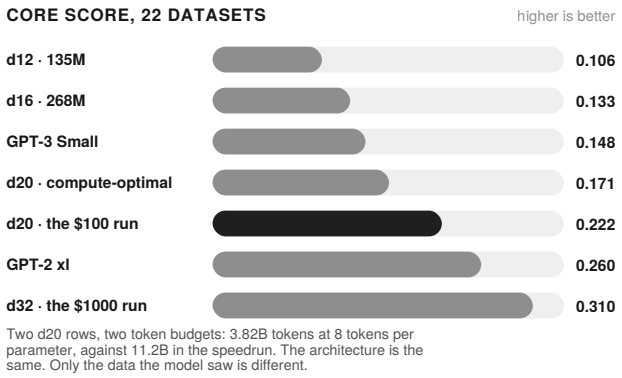


Fig. 6. CORE across the published runs and two GPT baselines. The two d20 rows share an architecture and differ only in how many tokens the model saw.

At depth 20 the compute-optimal run reaches CORE 0.1708 on 3.82B tokens; the overtrained speedrun reaches 0.2219 on 11.2B. Compute-optimal means optimal for a fixed training budget, not optimal for the model you intend to deploy. If the model is going to be served, the extra tokens are usually worth it, because inference cost is set by parameter count and parameter count is now fixed.

> **THE DISTINCTION WORTH KEEPING.** Chinchilla answers "given this much compute, what is the best model I can train". Deployment asks "given this model size I have to serve, how much should I train it". Those have different answers, and at small scale the gap is large enough to see in the benchmark.

VIII. MEASURING PROPERLY

The miniseries makes a methodological argument that is easy to skip and shouldn't be. Validation loss is not comparable across models trained on different data distributions, which makes it useless for the comparison everyone actually wants to make. nanochat uses CORE, averaged across 22 datasets, precisely so that a run can be placed next to GPT-2 and GPT-3 honestly.

This is the same discipline that distinguishes a serious evaluation from a demo in agent work, and it is cheap to adopt. A number that only compares a model to itself is a debugging tool. A number that compares it to something you did not train is a result.

The report card also makes the regressions visible. ARC-Challenge goes down after supervised finetuning, from 0.2875 to 0.2807. A pipeline that only reported its improvements would have hidden that. Publishing the full card, including the one row that got worse, is what makes the rest of the card believable.

That regression is also substantively interesting rather than noise to be apologised for. Supervised finetuning teaches a model to produce the kind of answer the demonstrations contain, and the demonstrations are conversational. ARC-Challenge rewards something closer to careful reasoning over a hard multiple-choice item. Making a model more helpful and making it more careful are not the same

objective, and at this scale there is not enough capacity to have both for free.

The general lesson is that a report card with a single headline number tells you nothing about the trade you made. A card with seven rows tells you what you gave up, and at small scale you always gave up something.

IX. THE LADDER, AND WHERE IT BREAKS

Because the dial is continuous, the cost of any capability level can be read off directly. The miniseries publishes the extrapolations.

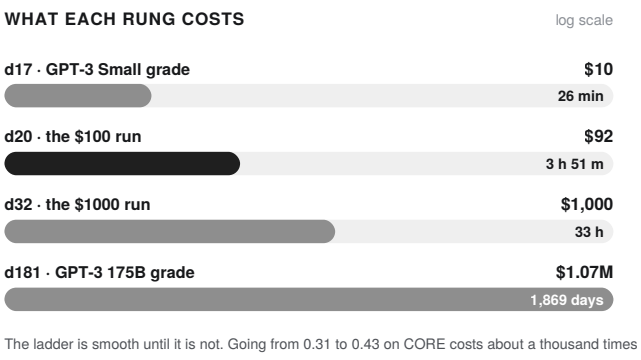


Fig. 7. Four rungs on the same ladder, on a log scale. The jump from GPT-2 grade to GPT-3 175B grade is about three orders of magnitude in both money and time.

TABLE IV
THE PUBLISHED LADDER

Target	Depth	Time	Cost
GPT-3 Small grade (0.148 CORE)	d17	~26 min	~$10
The speedrun (0.222 CORE)	d20	3 h 51 m	$92.40
Above GPT-2 (0.31 CORE)	d32	~33 h	~$1,000
GPT-3 175B grade (0.427 CORE)	d181	~1,869 days	~$1.07M

The depth 32 run is the one worth dwelling on. Karpathy reported that after about 33 hours it reached a CORE of 0.31, comfortably above GPT-2 at roughly 0.26, with metrics improving across pretraining, SFT and RL, and GSM8K moving from around 8% to around 20%. One thousand dollars buys a model that beats the 2019 state of the art.

The last row is the sobering one. Matching GPT-3 175B on CORE, by this extrapolation, needs depth 181, about 1,869 days of single-node time, and roughly $1.07 million. The ladder is perfectly smooth, and it is also perfectly brutal: the curve is logarithmic in capability and linear in spend.

There is a second reading of that number that is more useful than despair. GPT-3 175B grade is now a known price. It is not a mystery, not a moat and not a research question. It is a line item, and the question becomes whether the capability is worth it for your task, which is a question you can actually answer.

The extrapolation also explains, in one line, why the industry looks the way it does. If capability is logarithmic in spend, then the only entities that can move the frontier are the ones for whom a thousand-fold increase in budget is an ordinary decision. Everyone else is on the lower rungs by arithmetic, not by ambition.

Which is why the lower rungs deserve more attention than they get. The distance from $10 to $1,000 covers GPT-3 Small grade to comfortably past GPT-2, and that range is where almost every narrow production task actually sits. The expensive part of the curve is where general capability lives; the cheap part is where most jobs live.

X. AUTORESEARCH: HANDING OVER THE DIAL

In March 2026 Karpathy posted a 630-line repository that closes the loop. The idea: if a training run takes five minutes, a human is no longer the right thing to be deciding what to try next.

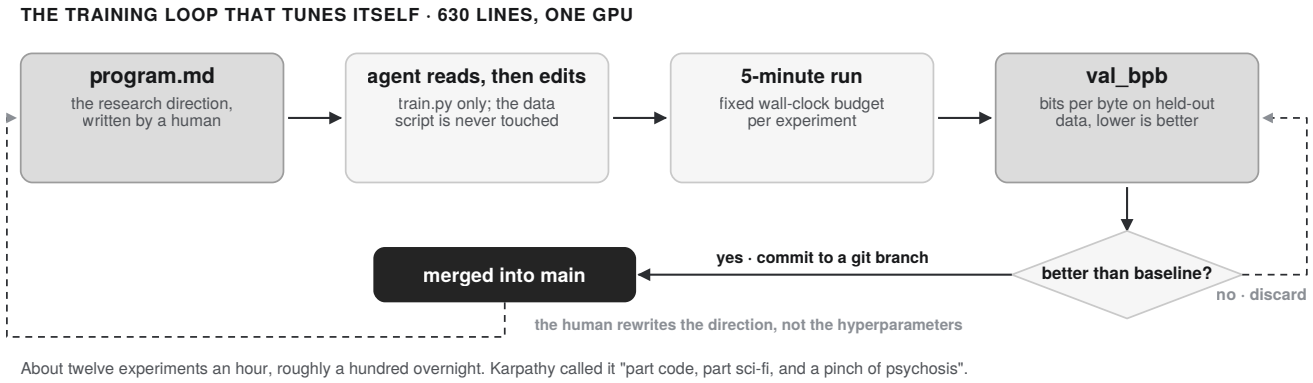


Fig. 8. The autoresearch loop. A human writes the research direction in prose; the agent edits the training script, runs a fixed-budget experiment, and keeps the change only if the held-out metric improves.

The setup has three files and a clear boundary. `prepare.py` handles data and is never touched by the agent. `train.py` holds the architecture and hyperparameters, and the agent may change it freely. `program.md` is the research direction, written by a human in prose.

```
# program.md  (the human writes this)
Goal: lower val_bpb at fixed wall clock.
Try: attention variants, lr schedules,
     init scaling, data ordering.
Do not: change the eval or the budget.
```

Each experiment gets a fixed five-minute wall-clock budget and is scored on validation bits per byte. Better than baseline, the change is committed to a git branch. Worse, it is discarded. About twelve experiments run per hour, roughly a hundred overnight, on a single H100. Karpathy described the repository as "part code, part sci-fi, and a pinch of psychosis".

Secondary coverage of early runs reports an agent making 110 code modifications over twelve hours and moving validation loss from 0.862415 to 0.858039 at unchanged training time, and separately reports the time to a GPT-2-grade model falling from roughly three hours to roughly two. Those figures come from third-party write-ups rather than the repository, and should be read as indicative rather than as a benchmark.

> **What actually changed.** The human stopped writing hyperparameters and started writing research directions. The loop is not automating the science. It is automating the sweep, which is the part of the science that was never interesting.

The reason this is possible at all is everything in the previous sections. The dial makes the search space one-dimensional where it can be. The staged pipeline makes a single stage cheap to isolate. The held-out metric makes "better" mean something. Hand an agent a codebase without those three properties and it optimises noise.

XI. What a Small Model Is Actually For

A 561-million-parameter model that gets 4.55% on GSM8K is useless as a general assistant and genuinely useful at several specific jobs. Conflating those two statements is the most common mistake in this area.

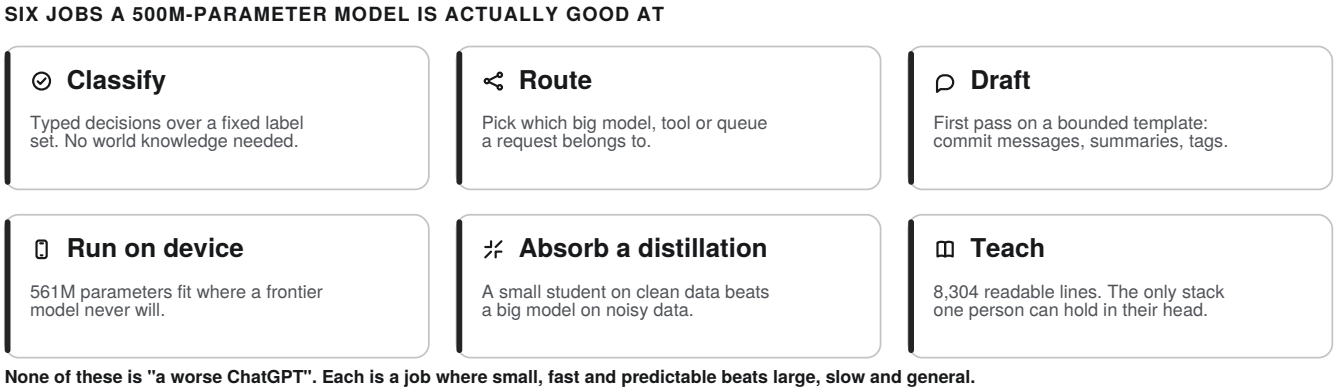


Fig. 9. Six jobs where small, fast and predictable beats large, slow and general. None of them is a general assistant, and that is the point.

Classification is the clearest case. A bounded label set needs no world knowledge, and a small model that answers in milliseconds for a fraction of a cent beats a frontier call that does the same job slower, more expensively, and with more variance. Routing is the same shape: deciding which model should handle a request is a smaller problem than handling it.

On-device is the structural one. 561M parameters fit somewhere a frontier model never will, and the cognitive-core argument in Section II is ultimately an argument about where the model runs. A model that lives always-on on a personal machine has a different cost structure from one behind an API, and that difference does not shrink with better hardware.

And then there is the use Karpathy built the repository for. 8,304 lines is the only full-stack language model pipeline that one person can read end to end. As an artefact for understanding how these systems work, it has no competition at any size.

XII. FOUR WAYS TO GET ONE

Everything up to here has been about training a model from nothing, because that is what the repositories teach. In practice almost nobody should start there. There are four routes to having a small model that does your job, they cost wildly different amounts, and the right one is decided by what you already have rather than by what you would enjoy building.

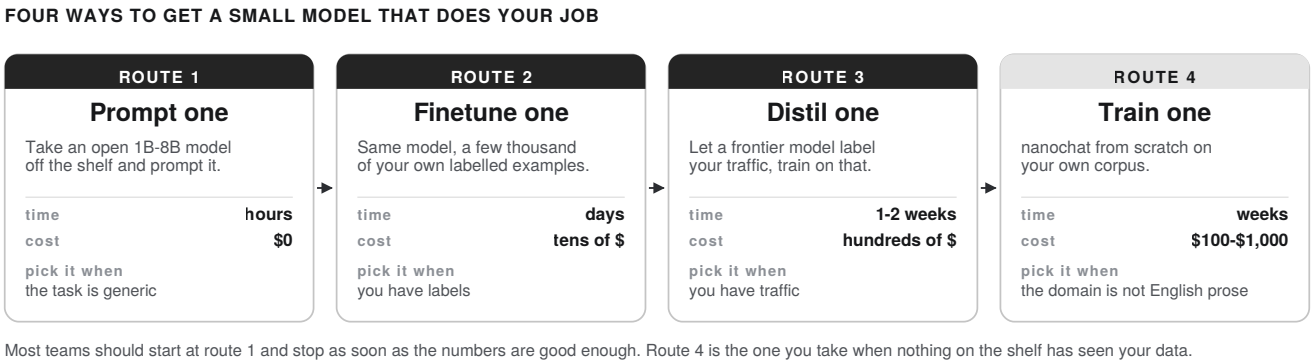


Fig. 10. Four routes, in increasing order of effort. The right one is whichever is the cheapest that clears your bar, and you only find that out by trying them in this order.

A. Prompt one off the shelf

The 2026 shelf of open models is good enough that for a generic task this is often the whole project. Pull a 3B or 8B model, write the prompt, measure it on your held-out set. If the number clears your bar, you are done, and you have spent an afternoon.

The trap here is measuring on vibes. A small model that looks fine on the five examples you tried by hand will fail on the long tail in a way a frontier model does not, because it has less slack. Build the held-out set first.

B. Finetune one

If the task has a shape the base model does not know, and you have labelled examples, a parameter-efficient finetune is days of work and tens of dollars of compute. What changes is format adherence and domain vocabulary. What does not change is world knowledge: finetuning teaches a model how to answer, not what it does not know.

C. Distil one

The route most production teams should take and most do not. You already have traffic. A frontier model can label it once, and a small model can be trained on the result. Section XIV is the recipe.

D. Train one from scratch

Justified when the shelf has not seen your data: a non-English corpus with no good coverage, a specialised symbolic domain, an internal format that looks nothing like web text, or a hard constraint that nothing with an unknown training set may touch your inputs. This is where nanochat stops being a teaching repository and starts being a tool.

TABLE V

WHICH ROUTE, BY WHAT YOU ALREADY HAVE

You have	Route	What it costs you
A generic task	Prompt an open model	An afternoon and a held-out set.
A few thousand labels	Finetune	Days, tens of dollars, a GPU.
Production traffic	Distil	One teacher pass over your logs, then a finetune.
A corpus nothing has seen	Train	Weeks and $100 to $1,000 of compute.
None of the above	Keep prompting the big one	Nothing, which is sometimes the right answer.

XIII. THE 2026 SHELF

The practical argument for small models stopped being theoretical when the open releases caught up. What follows is a sample of what was publicly available by mid-2026, reproduced from a published survey rather than measured here.

TABLE VI

A SAMPLE OF OPEN SMALL MODELS, AS SURVEYED

Model	Params	Context	Licence
Qwen3 family	0.6B – 8B	32K – 128K	Apache 2.0
Gemma 4 (E2B, E4B)	2B, 4B	up to 256K	Open weights
Ministral 3	3B, 8B	256K	Apache 2.0
Phi-4 mini	3.8B	model-dependent	Open weights
Granite 4.1	3B, 8B	model-dependent	Open models
SmolLM3	3B	64K trained, 128K supported	Apache 2.0
Tiny Aya	3.35B	8K	CC-BY-NC-4.0

Two things are worth reading off that table. The first is that the permissive licences are real: Apache 2.0 on a 3B model means the weights are yours, they do not change under you, and nothing leaves your network.

That is a different product from an API, not a cheaper version of one.

The second is the context windows. A 3B model with a 128K window is a different proposition from the 561M model in Section VI, because a large window partly substitutes for knowledge in the weights. The cognitive-core argument in Section II is exactly this: do not memorise, look it up. A long context is the cheapest form of looking it up.

> **Licence before benchmark.** A model you cannot deploy under your own terms is an API with extra steps. Check the licence before you spend a week on the evaluation.

XIV. Distilling the Model You Already Pay For

Distillation in production is not the research version. You are not matching logits or chasing a general student. You are taking one narrow job that a frontier model currently does ten thousand times a day, and moving it onto something you own.

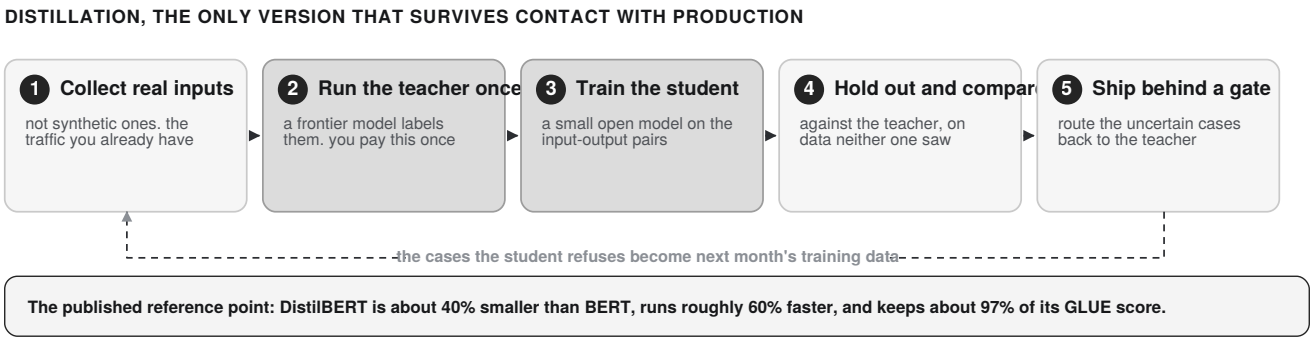


Fig. 11. The five-step distillation loop. The teacher runs once over real traffic; the student is trained on the result; the cases the student is unsure about go back to the teacher and become the next training set.

Collect real inputs. Not synthetic ones. The distribution that matters is the one your users produce, including the malformed, the truncated and the ambiguous. Synthetic data produces a student that is excellent on the data you imagined.

Run the teacher once. This is the only expensive step and you pay it a single time. Keep every output, including the ones you disagree with, because they are what the student will be scored against.

Train the student. A small open model on the input-output pairs. For a bounded task this is a parameter-efficient finetune, not a from-scratch run.

Hold out and compare. Against the teacher, on data neither model saw. The question is not whether the student is good. It is how much of the teacher it kept, and on which inputs it did not.

Ship behind a gate. The student answers when it is confident and escalates when it is not. This is what makes the whole thing safe to deploy: the failure mode is a slower answer, not a wrong one.

The published reference point for how much survives this process is DistilBERT, which is about 40% smaller than BERT, runs roughly 60% faster, and retains about 97% of its GLUE score. That is a 2019 result on a different architecture and a different task family, and it should be read as an existence proof rather than as a number you will reproduce.

The loop in Fig. 11 is the part that matters long-term. Every case the student escalates is, by construction, a

case the current training set did not cover. Collecting those and retraining on them monthly is a flywheel, and it is the reason a distilled model gets better in production while a prompted one does not.

XV. WHERE IT ACTUALLY RUNS

The reason to care about parameter count is not elegance. It is that parameter count decides where the model can live, and where it lives decides what it costs and who can see the data.

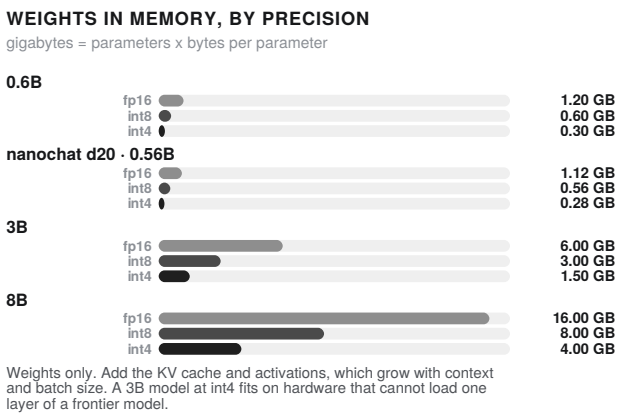


Fig. 12. Weights in memory, by precision. The arithmetic is parameters multiplied by bytes per parameter: two for fp16, one for int8, half for int4.

The arithmetic is simple enough to do in your head and consequential enough to decide an architecture. A 3B model at fp16 needs about six gigabytes for weights alone; quantised to int4 it needs about one and a half, which fits on a consumer GPU, a laptop, or a reasonably recent phone. The nanochat d20 model at int4 is under three hundred megabytes.

TABLE VII

WHAT EACH PRECISION COSTS YOU

Precision	Bytes/ param	3B model	What you give up
fp16 / bf16	2	~6.0 GB	Nothing. The reference.
int8	1	~3.0 GB	Usually very little on a narrow task.
int4	0.5	~1.5 GB	Measurable. Measure it on your task, not on MMLU.

Two caveats belong next to those numbers. Weights are not the whole memory bill: the KV cache grows with context length and batch size, and on a long-context workload it can exceed the weights. And quantisation damage is task-dependent. A model that loses two points of MMLU at int4 may lose nothing at all on a ten-label classifier, or it may fall apart on structured output. The only way to know is your own held-out set.

What this buys is the thing the API cannot sell you: a model running inside your network, at a cost that does not scale with how much you use it, with latency set by your hardware rather than by somebody else's queue.

XVI. THE REPLACEMENT MATH

The argument for owning a model is almost never about quality. It is about what happens to the bill when the volume goes up.

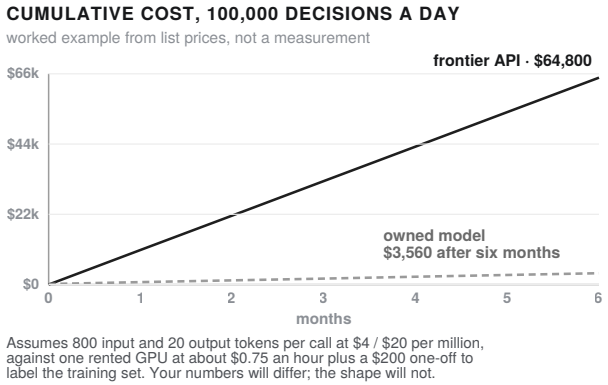


Fig. 13. Cumulative cost over six months at a hundred thousand decisions a day. A worked example from list prices with the assumptions stated, not a measurement.

Take a classification call with roughly 800 input and 20 output tokens, priced at $4 per million input and $20 per million output. That is about $0.0036 a call. At ten thousand calls a day it is roughly $36 a day, or about $1,080 a month. At a hundred thousand it is about $10,800 a month.

Now price the alternative. One rented GPU capable of serving a quantised 3B model, at roughly $0.75 an hour, is about $540 a month and does not care whether you send it ten thousand requests or a hundred thousand. Add a one-off of a couple of hundred dollars for the teacher pass that builds the training set.

TABLE VIII

A WORKED EXAMPLE, ASSUMPTIONS STATED

	10,000 / day	**100,000 / day**
Frontier API, per month	~$1,080	~$10,800
Owned model, per month	~$540	~$540
One-off to distil	~$200	~$200
Payback	~1 month	~2 days
Six-month total, API	~$6,480	~$64,800
Six-month total, owned	~$3,440	~$3,440

Arithmetic from published list prices and typical GPU rental rates, not a measurement. Serving capacity, utilisation, engineering time and the cost of being wrong are all excluded. The purpose is the shape of the curve, not the figures.

The honest reading of that table is that at low volume the saving is real but unexciting, and at high volume it is the difference between a line item and a problem. Below a few thousand calls a day, the engineering time to own a model costs more than the API does, and you should keep paying.

> **THE THRESHOLD QUESTION.** Not "is the small model as good". It is "at what volume does the recurring bill exceed the one-off cost of not having one". Below that line, rent. Above it, own.

There is a second cost that does not appear in either column and often decides the matter anyway. A frontier API is a dependency that reprices, deprecates and changes behaviour on somebody else's schedule. A model in your own registry does none of those things. For a decision sitting in the middle of a production path, that stability is frequently worth more than the money.

XVII. WHERE A SMALL MODEL MEETS AN AGENT

The reason this subject has stopped being academic is that agent systems made the shape of the demand obvious. An agent loop is mostly not reasoning. It is a long sequence of small, bounded decisions: is this urgent, how hard is it, which tool applies, was that output any good, is this command safe to run. Each of those is answered today by a frontier model call, and almost none of them needs one.

A 561-million-parameter model trained on your own traffic is a plausible answer to several of those. It will not plan a migration. It will decide, in milliseconds and for a fraction of a cent, which of eleven queues a ticket belongs to, and it will do it the same way twice, which is more than can be said for a sampled generation.

The structural argument runs through cost. Inference cost scales with parameter count, and parameter count is fixed at training time. A decision made ten thousand

times a day by a frontier model is a recurring bill that grows with your traffic. The same decision made by a model you trained once is a fixed asset with a flat marginal cost.

The structural argument against is capacity. Every table in this document says the same thing: at this scale there is no general knowledge to draw on and very little reasoning to spare. MMLU at 0.3151 is barely above chance. If the decision requires knowing something about the world that is not in the input, a small model will answer confidently and wrongly.

> **The dividing line.** If the answer is a function of the input, a small model can learn it. If the answer requires knowledge that was supposed to be in the weights, it cannot. Most routing, triage and gating decisions are the first kind, which is why they are the ones worth moving.

This is the same conclusion the cognitive-core argument reaches from the other direction. Facts live outside the model; the model looks them up. A small model with retrieval and tools is a different proposition from a small model alone, and it is the only version of the small-model argument that survives contact with a real task.

XVIII. ANTI-PATTERNS

Table IX

Nine ways a small-model project goes wrong

Anti-pattern	Why it bites
Benchmarking on validation loss	Not comparable across data distributions. Use CORE or your own held-out set.
Skipping midtraining	The base model completes your prompt instead of answering it, and SFT cannot fix that.
Chinchilla by reflex	8:1 is compute-optimal; 20:1 is deployment-optimal. Decide which you are doing.
Expecting general ability	MMLU 0.3151 is near chance. Pick jobs that need no world knowledge.
Tuning before measuring	Without a held-out metric, an automated sweep optimises noise very efficiently.
Only publishing improvements	ARC-Challenge regressed after SFT. A card with no regressions is not a card.
Training before cleaning	Model size is a data-quality problem. Dirty corpus, big model, every time.
Training before prompting	Route 4 before route 1. Try the shelf first; most tasks stop there.
Owning at low volume	Below a few thousand calls a day the engineering time costs more than the API.

XIX. A BUILD PLAN

There are two plans here, and which one you need depends on whether you are trying to understand this or to ship it. They are not alternatives; the first one makes the second one go faster.

A. The learning track

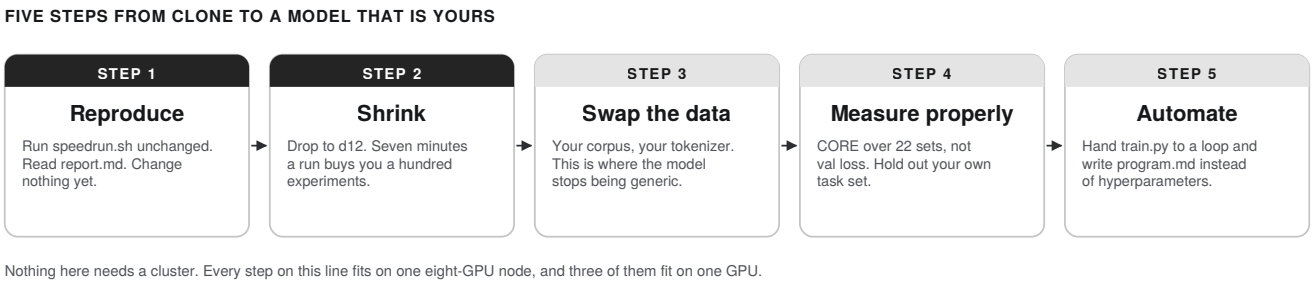


Fig. 14. Five steps, in order of what they teach. The first three cost less than a hundred dollars in total and none of them needs a cluster.

Step 1, reproduce. Run the speedrun unchanged and read the generated report. You are not learning the model here; you are learning the shape of the pipeline and what a normal report card looks like.

Step 2, shrink. Drop to depth 12. A run that finishes in under eight minutes changes what kind of question you are willing to ask, which is worth more than the compute you save.

Step 3, swap the data. Your corpus, your tokenizer. This is the step where the model stops being a teaching exercise and starts being yours, and it is also the step where you find out how much of your data is actually usable.

Step 4, measure properly. CORE for comparability, plus a held-out set from your own task. If you cannot state what number would make you ship, you are not ready to tune.

Step 5, automate. Hand the training script to a loop and write the research direction in prose instead of hyperparameters. Do this last, because it only works once the previous four steps hold.

B. The shipping track

Week 1. Pick the one decision your system makes most often and log a thousand real inputs with the frontier model's answers. You now have a training set and a baseline at the same time.

Week 2. Build the held-out set and label two hundred of them by hand. This is the only irreplaceable work in the whole plan, and skipping it invalidates everything after it.

Week 3. Prompt a 3B model from the shelf. Measure. If it clears the bar, ship it behind a confidence gate and stop reading.

Week 4. If it does not clear the bar, finetune on the logged pairs and measure again. This is where most tasks land.

Week 5. Quantise, measure the damage on your own set rather than on a public benchmark, and size the hardware from the arithmetic in Section XV.

Week 6. Ship behind the gate, route the uncertain cases to the teacher, and collect them. Those cases are next month's training data, and that loop is the whole point.

XX. LIMITATIONS

The repository is a teaching artefact, and Karpathy says so. It is deliberately readable rather than optimised, and nothing here should be read as a recipe for a competitive production model.

The extrapolations are extrapolations. Depth 181 at 1,869 days has not been run by anyone, and scaling curves fitted over d10 to d20 carrying all the way to GPT-3 175B is a long way outside the fitted range. The number is useful as an order of magnitude and nothing more.

The autoresearch results reported in secondary coverage come from short runs on one metric. Validation bits per byte moving in the fourth decimal place is a real improvement and also a small one, and a loop that optimises a single held-out number will eventually find ways to move that number that you did not intend.

And the central thesis remains unproven in the direction that matters. The cognitive-core argument says a clean corpus would let a far smaller model carry the same capability. Nobody has the clean corpus. Until somebody does, every small model is working with the same noisy data as the large ones, with fewer parameters to absorb it.

There is also a selection effect worth naming in the report card itself. The benchmarks nanochat reports are the benchmarks the field has standardised on, and they were designed to discriminate between large models. A 561M-parameter model scoring near chance on MMLU

tells you it has no encyclopedic knowledge, which we already knew. It tells you almost nothing about how the same model would perform on a narrow classification task with ten thousand labelled examples from your own domain, which is the use case that matters.

Finally, none of this addresses serving. The pipeline ends at a model and a chat interface. Everything a production deployment needs after that point, from batching and quantisation to monitoring and rollback, is outside the scope of the repository and is not made easier by the model being small.

XXI. CONCLUSION

The useful thing Karpathy built is not a model. It is a floor under the whole subject. Before nanochat, "what does it take to train a chat model" was a question whose answer depended on who you asked and what they were selling. Now it is 8,304 lines, four stages, one integer, and $92.40.

Everything above that floor becomes a decision rather than a mystery. GPT-2 grade costs a thousand dollars. GPT-3 175B grade costs about a million. A model that only has to classify, route or draft does not need either.

The method underneath is more portable than the models it produces. Collapse the dimensions you understand into one dial. Stage the pipeline so each step has a job you can name. Measure against something you did not train. Publish the regressions. Automate the sweep only after the other four hold. None of that is specific to language models, and all of it is unusual enough in practice to be worth copying.

And the reason to train the small one yourself is not that it will be better than the frontier model. It is that having done it once, you will know which of the thousand calls in your system were never a frontier problem at all.

ACKNOWLEDGMENT AND SOURCES

This document is an independent synthesis prepared for study. It is not affiliated with, reviewed by, or endorsed by Andrej Karpathy, Eureka Labs or OpenAI. All diagrams are original work produced for this paper. Quoted metrics retain the framing of their source.

[1] A. Karpathy, "Introducing nanochat: The best ChatGPT that $100 can buy," karpathy/nanochat Discussion #1, October 2025. Source for the four-stage pipeline, the $92.40 / 3 h 51 m speedrun, the depth-20 architecture, the tokenizer figures, the 8,304-line count and the full report card.

[2] A. Karpathy, "nanochat miniseries v1," karpathy/nanochat Discussion #420, January 2026. Source for the eleven-model series, the per-depth table, the 8:1 data-to-parameter ratio, the CORE methodology and the d17 and d181 extrapolations.

[3] A. Karpathy, "$1000 tier nanochat run," karpathy/nanochat Discussion #8, and the accompanying public post, October 2025. Source for the depth-32 run: ~33 hours, CORE 0.31 and GSM8K ~8% to ~20%.

[4] A. Karpathy, autoresearch repository and public post, 7 March 2026. Source for the 630-line loop, the three-file structure, the five-minute experiment budget, val_bpb as the objective, and the quoted description of the project.

[5] Secondary coverage of early autoresearch runs (Analytics Vidhya, Launchberg, Blockchain.news), March 2026. Source for the 110-modification run, the 0.862415 to 0.858039 validation figure and the reported time-to-GPT-2 reduction. Labelled as indicative in the text.

[6] A. Karpathy, public post on the LLM "cognitive core," 2025, and subsequent podcast remarks. Source for the few-billion-parameter target, the always-on multimodal tool-using description, the one-billion floor and the "no clean internet" constraint.

[7] J. Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022, for the 20:1 ratio referenced in Section VII.

[8] Turing Post, "Small Language Models in 2026: 10 to Know, Compared," 2026. Source for the open-model survey reproduced in Table VI, including parameter counts, context windows and licences.

[9] V. Sanh et al., "DistilBERT, a distilled version of BERT," 2019, for the 40% smaller / 60% faster / ~97% of GLUE reference

point in Section XIV, as restated in contemporary coverage of distillation practice.

Figures 2, 3, 4, 8, 9, 10, 11 and 14 are structural diagrams drawn for this paper and contain no measured data. Figures 5, 6 and 7 plot the published values in [1], [2] and [3] without modification. Figures 12 and 13 are arithmetic from the assumptions stated in the text, not measurements.

APPENDIX A: GLOSSARY

TABLE X
TERMS USED IN THIS PAPER

Term	Meaning
Depth	The number of transformer layers, and in nanochat the single dial that sets width, learning rate, token budget and runtime.
CORE	An averaged score across 22 datasets, used so that runs on different data distributions can be compared.
ChatCORE	The conversational counterpart, scored after the model can hold a turn.
val_bpb	Validation bits per byte. The objective autoresearch optimises. Lower is better.
Midtraining	The stage that adapts a base model to conversation format, multiple choice and tool calls.
SFT	Supervised finetuning on curated demonstrations. Buys quality, not capability.
Compute-optimal	The best model for a fixed training budget, which is not the best model for a fixed serving budget.
Cognitive core	A small, always-on, tool-using model that sacrifices encyclopedic recall for capability.
Speedrun	The single script that runs all four stages end to end and writes a report card.
Report card	The published table of metrics after each stage, regressions included.
Distillation	Training a small student on a large teacher's outputs over your own real inputs.
Quantisation	Storing weights at lower precision. int4 is a quarter of the memory of fp16.
KV cache	Stored attention state. Grows with context and batch, and is not counted in the weights figure.
Confidence gate	The rule that lets the small model answer when sure and escalate when not.

APPENDIX B: BEFORE YOU TRAIN ANYTHING

Ten questions. If any answer is missing, the run will produce a number nobody can interpret.

TABLE XI
TEN CHECKS BEFORE THE FIRST RUN

#	Check	Pass condition
1	Do you know what the model is for?	A named job from Fig. 9, not "an assistant".
2	Is there a held-out set from your task?	Built before training, never touched during it.
3	Is CORE or an equivalent in the loop?	Comparable to something you did not train.
4	Have you picked a ratio on purpose?	8:1 for a research run, 20:1 or more for one you will serve.
5	Do you know your serving budget?	Parameter count is fixed at training time and billed forever.
6	Is midtraining in the plan?	If not, expect a completion engine, not a chat model.
7	Is there a verifier for RL?	No verifier, no RL stage. Skip it honestly.
8	Is the corpus clean?	Model size is set by data quality more than by ambition.
9	Will you publish regressions?	A card with no regressions has not been read carefully.
10	Can one person read the pipeline?	If not, nobody will know which part caused the number.

The first and the last are the ones that decide the project. Everything in between is engineering.

APPENDIX C: A READING ORDER

If the goal is to understand how these systems work rather than to ship one, the repositories are a curriculum and the order is not optional.

TABLE XII

THE NANO SERIES AS A COURSE

Read	You will understand
micrograd	What a gradient is, mechanically, with nothing hiding behind a framework call.
nanoGPT	What pretraining is, and why a pretrained model is not a chatbot.
microgpt	That the whole thing fits in 200 lines once you stop needing it to be fast.
nanochat — speedrun	The shape of a full pipeline, and what a report card is supposed to contain.
nanochat — miniseries	How scaling laws turn into a dial, and why CORE beats validation loss.
autoresearch	What is left for a human once the sweep is automated.

The whole sequence is a few thousand lines. It is the only route from "a language model is a thing that predicts the next token" to "I know exactly what each of the four stages cost me and what each one bought" that does not require a research budget.

And it is worth stating the obvious at the end. None of this makes a small model competitive with a frontier one, and nothing in the repositories claims it does. What it does is remove the mystery. After an afternoon and ninety-two dollars, the question "could we just train one" stops being rhetorical, and starts being a number you can put next to the invoice you are already paying.